CSD457 INTERNET OF THINGS · GROUP G-04 · OCTOBER 2026

# Progress report: Poison-Resilient On-Device Intrusion Detection for Drifting IoT

Rishit Kamboj and Farhan Naik · What has changed since the proposal deck, what is done, and exactly what we will build, test and measure next.

## The project in one paragraph

An IoT device that learns what normal traffic looks like can be taught, slowly, that attacks are normal. We are building a detector that **keeps learning on an ESP32** and cannot be steered that way. Our first method rationed only the part of each update that moves toward a known attack. New 2026 papers prove that a detector watching a single stream cannot reliably separate honest change from slow poisoning, so our next step gives each device a **second source of evidence: its neighbours**. Devices that share the same network see the same honest drift; an attacker usually reaches only one.

## Where we stand

| Area | Status | One line |
|---|---|---|
| **Idea and literature** | **Done** | 3 research rounds; 2024 to 2026 papers read; novelty checked against prior work |
| **Method v1: Threat-Orthogonal Adaptation (TOA)** | **Done** | Works on synthetic and on real N-BaIoT traffic |
| **Method v2: signature-free version** | **Done** | Beats a normal and a smart attacker on real traffic; fails when the attack hides inside strong honest drift |
| **Method v3: peer-witnessed adaptation (FOA)** | **Designed** | Mechanism and plan written; not yet implemented or tested |
| **Freeze attack on published defences** | **Partly** | Shown on our own baselines; not yet on published ones |
| **ESP32 hardware** | **Not yet** | No measurements yet. Parts list and wiring are in this report |
| **Report and code package** | **Partly** | Repository, notes and figures exist; IEEE report not started |

**In short:** The core idea is proven in simulation and on one real device's traffic. What remains is the part that makes it a research result: the peer mechanism, comparison with published defences, and real ESP32 measurements.

## Decisions we need from you

- How many ESP32 boards and INA219 modules we actually have, and whether we buy more.
- The real mid-review and final dates, so the weeks above can be pinned to them.
- Whether to keep the title or add the method name.

PART 1

# What changed since the proposal deck

The proposal deck (24 September) presented one method, TOA, backed by a synthetic simulation.

| | Proposal deck | Now |
|---|---|---|
| **Method** | TOA: ration movement toward a *known attack signature* | Signature-free version added (learns from the dense core of traffic; accepts a shift only if the outer ring backs it). Next: peer-witnessed version (FOA) |
| **Evidence** | Synthetic simulation only | Plus real N-BaIoT doorbell traffic and real Mirai attacks (scan, ack, syn); attacker is modelled |
| **Poisoning risk** | Estimated from the theory of attacker share | Measured on real traffic: naive learning fails at 10% attacker share (scan) or 20% (ack, syn) |
| **Main weakness** | Needs a signature for each attack | Removed by the signature-free version. New weakness: an attack hidden inside strong honest drift is still not stopped |
| **Understanding of that weakness** | Open question | Three 2026 papers prove it is a **limit for any single-stream detector**. The way out is extra evidence from outside the stream |
| **New devices** | Not considered | A device joining mid-stream gave about 34% false alarms for every detector. Fix: one detector per device that starts from its peers |
| **Novelty claim** | Peer-free defence | Corrected: ANTIDOTE (2009) already defends a server-side detector. Our claim is now narrower and defensible (see Part 3) |
| **Literature** | 5 core papers | 5 core plus 24 papers from 2024 to 2026, with prior-art checks on each candidate direction |

## Corrections we made along the way

- The claim that naive learning is poisoned at 10% attacker share held only for the Mirai scan attack. For ack and syn it takes 20%. All documents now say 10 to 20%.
- A claim that traffic fell 52% at semester end came from an outside analysis of the CESNET data, not from the CESNET paper itself. It should be worded accordingly or dropped.
- The novelty claim was narrowed after we found ANTIDOTE, and again after the 2026 impossibility papers.

PART 2

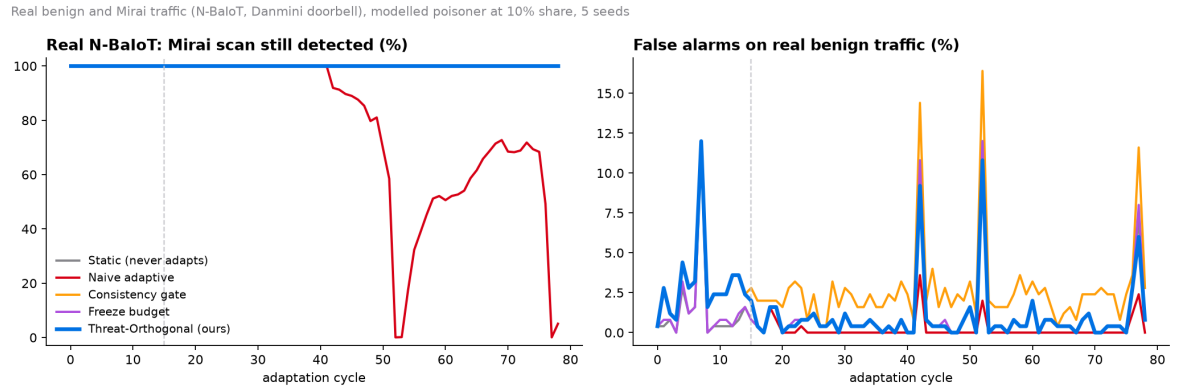
# Progress and results so far

## What we built

- **Detector:** unsupervised online centroid detector (centre and spread) on PCA-compressed Kitsune-style features. It labels its own traffic and updates every cycle.
- **Attacker:** boiling-frog poisoner that places poison just inside the alarm boundary toward a target attack. On real data each poison packet mixes a real normal packet with a real attack packet.
- **Defences compared:** static, naive adaptive, consistency gate, freeze budget, TOA, signature-free TOA.
- **Code: experiments/sim_toa.py**, **nbaiot_check.py**, **auto_toa.py** in the repository.

## Key results

| Test | Result |
|---|---|
| **Synthetic, attacker at 40% of traffic** | Naive: attack hidden after 24 cycles. Freezing defences: 85 to 87% false alarms. TOA: 100% detection, 0.3% false alarms |
| **Real doorbell traffic, boundary attacker** | Naive fails at 10% share (scan) and 20% (ack, syn). TOA and signature-free: 100% detection, about 0.8% to 1.5% false alarms, at 10 to 50% share |
| **Real traffic, smart attacker hiding poison in the core** | Signature-free version: 100% detection, 0.2% to 0.7% false alarms. A first attempt failed here; adding the outer-ring check on spread growth fixed it |
| **Synthetic, strong honest drift and attack together** | Signature-free version is still poisoned in three of the four settings tried (2% to 46% detection). This is the open problem |
| **New device joins (real thermostat traffic)** | All detectors about 34% false alarms even with no attacker: one shared self-learning detector cannot absorb a new device |




*Real N-BaIoT Danmini doorbell and Mirai scan traffic, modelled poisoner at 10% share, 5 seeds. The thick blue line (ours) stays at 100% detection while naive learning (red) loses the attack.*

## What the results do not show yet

- Anything on a microcontroller. All runs so far are Python on a laptop.
- The freeze attack on real long-term drift or on published defences.
- More than one device's traffic as peers. The peer mechanism has not been implemented.
- A real attacker. The poisoner is modelled because no public dataset contains poisoning.

PART 3

# What our research is now about

**In short:** Make a fleet of ESP32 detectors that keep learning honest change but cannot be steered by an attacker, by letting neighbouring devices act as witnesses, and measure what that costs on real hardware.

## Research questions

- **RQ1.** Can peer witnesses let a device keep learning honest drift while resisting poisoning, in cases where a single-stream detector provably cannot?
- **RQ2.** Do published defences that stop or gate learning fall to a freeze attack, and does never-stopping adaptation avoid it?
- **RQ3.** What does it cost on an ESP32 in memory, latency, radio bytes and energy, and how much does it slow legitimate adaptation?

## The method: Fleet-Orthogonal Adaptation (FOA)

Each device keeps its own centre-and-spread detector. Every cycle it proposes an update from the dense core of its own traffic and shares that small proposal (10 numbers, signed) with its peers over ESP-NOW. The peers' robust consensus (their geometric median) is the **common part**: network-wide drift such as a router change or a daily cycle. The rest is the **residual**.

- The common part is applied in full, so honest drift is learned and the detector never freezes.
- The residual goes through TOA's slow budget, so a campaign that only one device sees is rationed.
- Peers are chosen to differ in vendor or firmware, so one exploit cannot make several devices agree.
- A new device starts from its peers' median and learns under the residual budget only, which bounds how much an attacker can poison it while it joins.

## Why this is different from published work

| Closest work | What it does | How we differ |
|---|---|---|
| **CIoTA (NDSS DISS 2018, JPDC 2020)** | Same-firmware devices must agree before learning; disagreement is discarded | Our peers differ in firmware, so a one-exploit botnet cannot fake agreement; we admit the disagreeing part slowly instead of discarding it |
| **ANTIDOTE (IMC 2009)** | Robust PCA on a server, retrained in batches | Continuous learning on a microcontroller, with peers |
| **WARDEN/CDC, 2026 papers on poisoning theory** | Prove single-stream limits | We use a second stream to escape them, and must prove that as our own theorem |
| **DriftXpert, OWAD, T2T rollback (2023 to 2026)** | Stop or gate learning on a statistic | These are the freeze-attack targets; FOA has no stop gate |

*Every candidate direction came back partially covered in our prior-art checks, so we claim a specific combination rather than a first: a budgeted split between peer-explained and unexplained updates, tested against a monoculture-botnet attacker, with a freeze-attack study on published defences and ESP32 measurements.*

PART 4

# IoT components and set-up

The experiment needs a small fleet of real ESP32 devices on one network, an attacker, and a way to measure energy.

| Item | Qty | Why we need it |
|---|---|---|
| **ESP32 development board (for example ESP32-DevKitC)** | 4 minimum, 5 with a spare | Fleet nodes A to D and the power monitor. You said you have at least one; please check the count |
| **Temperature or humidity sensor (BME280 or DHT22)** | 1 per node (optional) | Gives each node real telemetry to send, so its traffic is genuine |
| **INA219 current-sensor module** | 1 or 2 | Measures a node's power and energy |
| **5 V USB supply or breakout board** | 1 | Feeds the node through the INA219 |
| **Breadboard, jumper wires, 4.7 kohm resistors** | a set | Wiring and I2C pull-ups |
| **USB cables and a powered USB hub** | 5 | Flashing and serial logs |
| **Wi-Fi router or phone hotspot on a fixed channel** | 1 | ESP-NOW and Wi-Fi must share one channel |
| **Laptop** | 1 | MQTT broker (Mosquitto), poisoner script, N-BaIoT replay, data logging |

## Software and tools

- ESP-IDF or Arduino core for ESP32, with ESP-NOW and mbedTLS (HMAC) from the standard SDK.
- Python with NumPy for experiments; Mosquitto for MQTT telemetry; the existing N-BaIoT experiment scripts.
- Two run modes: **replay mode** (features from N-BaIoT are fed to the board over serial, to measure compute cost reliably) and **live mode** (the board's own telemetry traffic).

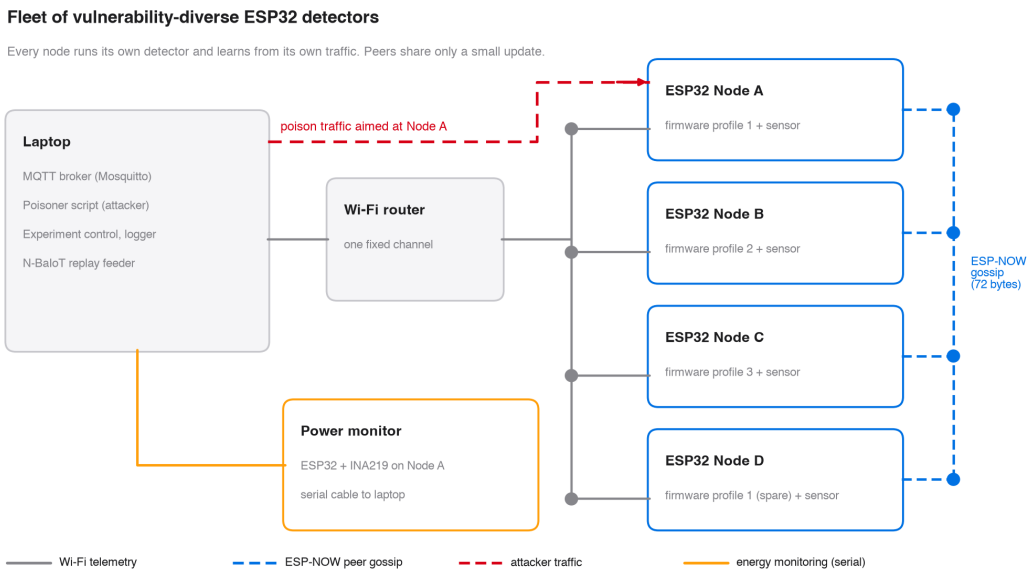
## What we will measure on the boards

- Memory: peak RAM and flash used by the detector, the peer step and the gossip buffer.
- Time: latency per update cycle, and time to verify one signed gossip message.
- Radio: bytes sent per learning event over ESP-NOW (about 72 bytes: 10 numbers plus a signature).
- Energy: millijoules per update cycle, from the INA219, reported separately for compute and for Wi-Fi.

## Why we expect it to fit

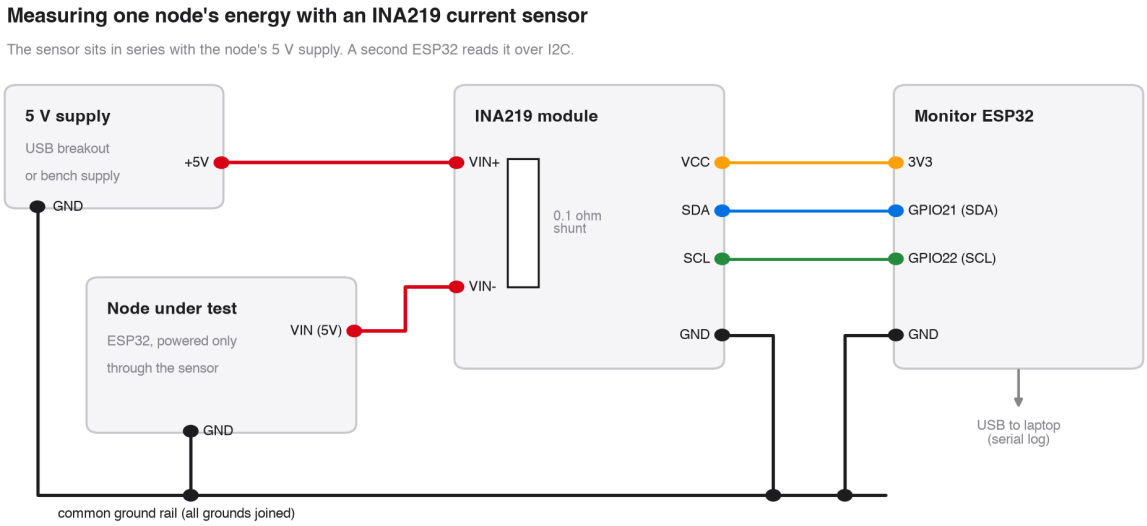
An ESP32 has two 240 MHz cores and about 520 KB of RAM. Per packet our detector needs about a thousand multiply-adds and under 10 KB of state; the peer step is a 10-dimensional median over a few neighbours. These are estimates, not measurements. Measuring them is research question RQ3.

## System layout




*Four ESP32 nodes with different firmware share a Wi-Fi network and gossip small signed updates over ESP-NOW. The laptop runs the MQTT broker, the poisoner and the logger. A separate ESP32 reads the INA219 on Node A.*

## Wiring for energy measurement




Build notes (check before wiring):
1. Add 4.7 kohm pull-ups from SDA and SCL to 3V3 if the INA219 module has none.
2. Flash the node first, then power it only through this path. Do not also plug in its USB at the same time.
3. INA219 samples about every 0.5 ms, so measure energy over many cycles and average. Do not trust single short bursts.

*Proposed wiring, not yet built. Check each pin against your module's datasheet before powering it. The INA219 is wired in series with the node's 5 V supply (high-side). Default I2C pins on the ESP32 DevKitC are GPIO21 (SDA) and GPIO22 (SCL).*

PART 5

# What we will test

Each test answers one claim. The headline result is the first two together.

| # | Test | Compare against | Measure | We win if |
|---|---|---|---|---|
| 1 | Attack hidden inside strong honest drift (the open problem) | Signature-free single-stream; naive; WARDEN-style frozen reserve | Detection and false alarms as drift and attacker share vary | FOA keeps detection where single-stream fails |
| 2 | Monoculture botnet: the attacker reaches several same-firmware peers | CIoTA-style same-firmware consensus | Time to compromise; detection | Diverse peers hold while same-firmware consensus is fooled |
| 3 | Freeze attack on published gated defences | Kloft and Laskov switch-off rule; T2T rollback; OWAD demo | False alarms after the attacker leaves; freeze duty cycle | Gated defences end near 85% false alarms; FOA stays near nominal |
| 4 | New device joins | One shared detector (34% false alarms earlier) | False alarms after join; join-time poisoning | False alarms near nominal |
| 5 | Cost of caution: vendor-specific honest drift | Naive; TOA alone | Adaptation lag; false alarms | Lag is small and measured |
| 6 | Adaptive attackers | Core-hiding, window-blind schedule, pulsing, omission | Time to compromise | No attacker beats the budget bound |
| 7 | ESP32 cost | Naive detector on the same board | RAM, latency, radio bytes, energy | Overhead small enough to justify |
| 8 | Real long-term drift (stretch) | As above | Detection against the theory ceiling | Ceiling lifted on real calendar drift |

## Data

- **N-BaIoT:** Danmini, Ecobee, Philips and the four cameras as peers. The devices were recorded separately, so shared drift is injected by us and labelled synthetic.
- **The ESP32 fleet:** a real router change gives genuine shared drift; a laptop script poisons one board.
- **CESNET-TimeSeries24:** real calendar drift. A stretch goal, not on the critical path.

## The one headline figure

Two panels. Left: under a monoculture botnet, same-firmware consensus is compromised while our diverse peers hold. Right: false alarms after the attacker leaves, with gated defences high and ours flat.

PART 6

# Plan of action

Seven working weeks. Adjust the start to the real mid-review and final dates, which we should confirm with Dr. Tapadar.

| Wk | Work | Output |
|---|---|---|
| **1** | Rebuild the N-BaIoT download as a script (the earlier files were temporary). Build the multi-device harness with shared and vendor-specific drift. Read the three unread papers | Reproducible data and harness |
| **2** | Implement FOA: geometric median, signed gossip, two cumulative budgets, join rule. Reuse the TOA code | FOA in simulation |
| **3** | Implement baselines: CIoTA-style, clipped robust aggregation, WARDEN-style, Kloft and Laskov rule, T2T. Run the freeze attack on them | Tests 3 and 4 |
| **4** | Attacker matrix and the headline figure. Tests 1, 2, 5, 6. Prepare the mid-review pack | First real results |
| **5** | Build the ESP32 fleet. Port the detector and FOA. Wire the INA219. First measurements | Test 7, hardware demo |
| **6** | Write the proofs (a standalone multi-device theorem with its negative case). Ablations: peer count, budget, diverse vs random peers | Theory and ablations |
| **7** | IEEE report, code package with README, live demo | Final deliverables |

## Mid-review checklist (from the course brief)

| Required | Status |
|---|---|
| **Clear problem statement and research question** | **Done** (RQs updated above) |
| **Literature table of recent papers** | **Partly**: 2024 to 2026 list exists; one merged table still to build |
| **Research gap and proposed contribution** | **Done** |
| **System architecture and workflow** | **Draft**: diagram in this report |
| **Baselines implemented or clearly defined** | **Partly**: four built; published ones defined, not built |
| **Experimental setup, data plan and evaluation metrics** | **Done** (Part 4 and 5) |
| **Initial results or a clear plan** | **Done** (synthetic and real-doorbell results) |

## Risks and fallbacks

| Risk | Fallback |
|---|---|
| **Peer concurrency on N-BaIoT is synthetic** | Let the real ESP32 router change and CESNET carry the real-drift claim |
| **The multi-device proof is harder than it looks** | Present it as a stated assumption plus the matching negative result |
| **Multi-device plumbing, ESP-NOW channel clashes, or slow on-board features** | Ship the simpler variant that learns threat directions from the detector's own alarms; use replay mode for cost; keep the freeze-attack study, which stands alone |
| **Novelty is partial; some 2026 papers are preprints** | Pitch the specific combination and win test 2; verify numbers against PDFs before citing |